

Fintech App — Documentação do Backend

Visão Geral

API REST desenvolvida em **Spring Boot 4.x** (Java 17) para gerenciamento financeiro pessoal. Permite que usuários registrem contas, categorizem transações, importem extratos bancários e acompanhem snapshots do patrimônio.

Stack: Spring Boot 4.0.5 · Spring MVC · Spring Data JPA · PostgreSQL · Lombok · Jakarta Validation · JUnit 5 · Mockito

Arquitetura Hexagonal (Ports & Adapters)

O projeto segue a arquitetura hexagonal: o domínio é isolado no centro e se comunica com o mundo externo apenas por meio de interfaces (portas).

ADAPTERS IN
(Controllers REST)

ADAPTERS OUT
(Persistence JPA)

APPLICATION (Use Cases)

DOMAIN (Models · Ports · Exceptions)

Regra de dependência

- `domain` não depende de nada — apenas Java puro.
 - `application` depende apenas de `domain` (usa as portas como interfaces).
 - `adapters` dependem de `application` e `domain`, nunca o contrário.
-

Estrutura de Pacotes

`com.enterprise.gustadev.fintech_app`

`domain/`

`<entidade>/`

`model/` — POJO com lógica de validação (`validar()`)

`port/` — Interface do repositório (ex.: `ContaFinanceiraRepositoryPort`)

`exception/` — Exceção de domínio específica

`application/`

`<entidade>/`

`usecase/` — Um arquivo por operação (Criar, Listar, Buscar, Deletar...)

`adapters/`

`in/web/`

`<entidade>/`

`<Entidade>Controller.java`

`dto/` — Records com `@Valid` para request e response

`out/persistence/`

`<entidade>/`

`<Entidade>Entity.java` — `@Entity` JPA com `fromDomain/toDomain`

`<Entidade>JpaRepository.java` — `extends JpaRepository<>`

`<Entidade>RepositoryAdapter.java` — implementa a porta do domínio

`config/`

`BeanConfig.java` — Registra todos os use cases como `@Bean`

Entidades do Domínio

Entidade	Tabela PostgreSQL	Descrição
----------	-------------------	-----------

ContaFinanceira	contas_financeiras	Conta bancária do usuário (corrente, poupança, cartão...)
Categoria	categorias	Categoria de gasto/receita, pode ser padrão ou personalizada
CategoriaThreshold	categoria_thresholds	Limite de gasto por categoria para alertas
Extrato	extratos	Arquivo de extrato bancário importado (PDF/CSV)
Transacao	transacoes	Lançamento financeiro individual
RegraClassificacao	regras_classificacao	Regra automática para categorizar transações
ClassificacaoLog	classificacao_logs	Histórico de classificações automáticas por IA
ProcessamentoJob	processamento_jobs	Job assíncrono de parsing de extrato
SnapshotFinanceiro	snapshots_financeiros	Foto mensal do patrimônio do usuário
AuditoriaEvento	auditoria_eventos	Log de auditoria de todas as operações
ConsentimentoLgpd	consentimentos_lgpd	Registro de aceite de termos (LGPD)
Notificacao	notificacoes	Alertas enviados ao usuário
ParserVersao	parser_versoes	Versão do parser de extrato por banco

Legado preservado: `Gasto` e `Usuario` usam `Long` como ID e têm estrutura independente.

Padrão de modelo de domínio

Todo modelo é um POJO com Lombok e um método `validar()` :

```
@Getter @Setter
public class ContaFinanceira {
```

```

private UUID id;
private UUID usuarioid;
private String nome;
private TipoConta tipo;
// ...

// Construtor conveniente (sem id, sem timestamps)
public ContaFinanceira(UUID usuarioid, String nome, TipoConta tipo,
                        String banco, BigDecimal saldoInicial, boolean padrao) { ... }

public void validar() {
    if (usuarioid == null) throw new ContaFinanceiraInvalidaException("Usuarioid é obrigatório");
    if (nome == null || nome.isBlank()) throw new ContaFinanceiraInvalidaException("Nome é obrigatório");
    // ...
}
}

```

Enums Compartilhados

Localizados em `domain/shared/enums/` :

Enum	Valores
TipoConta	corrente , poupanca , cartao_credito , investimento , carteira , outro
TipoCategoria	gasto , receita , transferencia
TipoTransacao	gasto , receita , transferencia
OrigemTransacao	extrato , manual , ia
StatusRevisaoTransacao	extraida , revisada , confirmada , rejeitada
StatusExtrato	upload_recebido , processando , concluido , erro
EstrategiaClassificacao	regex , ml , fuzzy , manual
CanalNotificacao	push , email , sms
TipoConsentimentoLgpd	termos_uso , politica_privacidade , compartilhamento_dados

Use Cases

Cada use case é uma classe com um único método `executar(...)` . Todos são registrados como `@Bean` em `BeanConfig` — não usam `@Service` .

Fluxo padrão

```
Controller  monta modelo de domínio  chama useCase.executar()
UseCase     chama modelo.validar()  chama repository.salvar()
RepositoryAdapter  converte domain entity  persiste via JpaRepository
```

Exemplo com regra de negócio — CriarExtratoUseCase

```
public Extrato executar(Extrato extrato) {
    extrato.validar();
    // Impede duplicata: mesmo arquivo não pode ser importado duas vezes
    repository.buscarPorHash(extrato.getHashArquivo()).ifPresent(e -> {
        throw new ExtratoInvalidoException("Extrato duplicado: hash já processado");
    });
    return repository.salvar(extrato);
}
```

API REST — Endpoints

Contas Financeiras `/contas`

Método	Rota	Descrição
POST	<code>/contas</code>	Criar conta
GET	<code>/contas/{id}</code>	Buscar por ID
GET	<code>/contas/usuario/{usuarioId}</code>	Listar contas do usuário
DELETE	<code>/contas/{id}</code>	Remover conta

Categorias /categorias

Método	Rota	Descrição
POST	/categorias	Criar categoria personalizada
GET	/categorias/{id}	Buscar por ID
GET	/categorias/padrao	Listar categorias padrão do sistema
GET	/categorias/usuario/{usuarioid}	Listar categorias do usuário

Extratos /extratos

Método	Rota	Descrição
POST	/extratos	Registrar extrato (com validação de hash anti-duplicata)
GET	/extratos/{id}	Buscar por ID
GET	/extratos/usuario/{usuarioid}	Listar extratos do usuário

Transações /transacoes

Método	Rota	Descrição
POST	/transacoes	Criar transação
GET	/transacoes/{id}	Buscar por ID
GET	/transacoes/usuario/{usuarioid}	Listar por usuário
GET	/transacoes/conta/{contaid}	Listar por conta
GET	/transacoes/extrato/{extratoid}	Listar por extrato
DELETE	/transacoes/{id}	Remover transação

Outros

Recurso	Rota base
Snapshots financeiros	/snapshots

Notificações	/notificacoes
Consentimentos LGPD	/consentimentos
Regras de classificação	/regras-classificacao

Padrão de request/response

Todas as rotas usam JSON. Campos obrigatórios são validados com `@Valid` no controller. Exemplos:

```
// POST /contas
{
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "nome": "Nubank",
  "tipo": "corrente",
  "banco": "Nubank",
  "saldoInicial": 1500.00,
  "padrao": true
}

// POST /transacoes
{
  "userId": "550e8400-...",
  "accountId": "661f9511-...",
  "tipo": "gasto",
  "valor": 89.90,
  "dataTransacao": "2025-05-22",
  "origem": "manual",
  "descricaoUsuario": "Supermercado Extra"
}
```

Camada de Persistência

Entidade JPA

Cada entidade JPA tem métodos estáticos `fromDomain` e `toDomain` para conversão:

```

@Entity @Table(name = "contas_financeiras")
@Getter @Setter @NoArgsConstructor
public class ContaFinanceiraEntity {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    // ...

    public static ContaFinanceiraEntity fromDomain(ContaFinanceira domain) { ... }
    public ContaFinanceira toDomain() { ... }
}

```

Repository Adapter

Implementa a porta do domínio delegando ao `JpaRepository` :

```

public class ContaFinanceiraRepositoryAdapter implements ContaFinanceiraRepository {
    private final ContaFinanceiraJpaRepository jpaRepository;

    @Override
    public ContaFinanceira salvar(ContaFinanceira conta) {
        return jpaRepository.save(ContaFinanceiraEntity.fromDomain(conta)).toDomain();
    }
}

```

Configuração

BeanConfig

Todos os use cases são instanciados manualmente como `@Bean` porque não usam `@Service` . Isso mantém o domínio e a aplicação livres de anotações Spring:

```

@Configuration
public class BeanConfig {
    @Bean
    public CriarContaFinanceiraUseCase criarContaFinanceiraUseCase(ContaFinanceiraRepository repository) {
        return new CriarContaFinanceiraUseCase(repository);
    }
}

```



```
// ... um @Bean por use case  
}
```

Perfis de ambiente

Perfil	Arquivo	Uso
dev	application-dev.yaml	Desenvolvimento local
hml	application-hml.yaml	Homologação
prd	application-prd.yaml	Produção

O perfil ativo é definido pela variável de ambiente `SPRING_PROFILE` (padrão: `dev`).

Testes

Estrutura (51 testes)

```
src/test/java/  
  domain/                — Testes de unidade dos modelos (validar())  
    contafinanceira/ContaFinanceiraTest.java  
    transacao/TransacaoTest.java  
    extrato/ExtratoTest.java  
    categoria/CategoriaTest.java  
  
  application/           — Testes de use cases com Mockito  
    contafinanceira/  
      CriarContaFinanceiraUseCaseTest.java  
      ListarContasFinanceirasUseCaseTest.java  
      BuscarContaFinanceiraUseCaseTest.java  
      DeletarContaFinanceiraUseCaseTest.java  
    transacao/  
      CriarTransacaoUseCaseTest.java  
      ListarTransacoesUseCaseTest.java  
    extrato/CriarExtratoUseCaseTest.java  
    categoria/CriarCategoriaUseCaseTest.java  
  
  adapters/in/web/       — Testes de controller com MockMvc
```

```
contafinanceira/ContaFinanceiraControllerTest.java
categoria/CategoriaControllerTest.java
transacao/TransacaoControllerTest.java
```

Tecnologias de teste

- **Domínio:** JUnit 5 + AssertJ — sem Spring, sem mocks
- **Use cases:** `@ExtendWith(MockitoExtension.class)` — `@Mock` no repositório, `@InjectMocks` no use case
- **Controllers:** `MockMvcBuilders.standaloneSetup()` — sem contexto Spring, mocks via `@Mock`
- **Integração:** `FintechAppApplicationTests` — sobe contexto completo com H2 em memória

Nota Spring Boot 4.x: `@WebMvcTest` e `@MockBean` foram removidos nesta versão. Use `standaloneSetup` para controllers e `@MockitoBean` (de `org.springframework.test.context.bean.override.mockito`) para testes com contexto completo.

Como rodar

```
# Todos os testes
./mvnw test

# Classe específica
./mvnw test -Dtest=ContaFinanceiraTest

# Método específico
./mvnw test -Dtest="CriarExtratoUseCaseTest#executar_deveLancarExcecao_quandoHa

# Build sem rodar testes
./mvnw package -DskipTests
```

No IntelliJ: botão  ao lado do nome da classe ou método, ou `Ctrl+Shift+F10`.

Como Executar Localmente

Pré-requisitos: Java 17, PostgreSQL rodando, variáveis de ambiente configuradas.

```
# Variáveis necessárias (perfil dev)
export DB_URL=jdbc:postgresql://localhost:5432/fintech
export DB_USERNAME=postgres
export DB_PASSWORD=sua_senha
```

```
# Subir a aplicação
./mvnw spring-boot:run
```

```
# Ou gerar o WAR e fazer deploy
./mvnw package
```

A aplicação sobe na porta `8080` . Endpoint de saúde: `GET /actuator/health` .